

Assignment No. 1

Title

DFS and BFS on undirected graph using recursive algorithm.

Aim

Implement depth first search algorithm and Breadth First Search algorithm, Use an undirected graph and develop a recursive algorithm for searching all the vertices of a graph or tree data structure..

Objectives

- To study the concept of Depth First Search & Breadth First Search.
- To implement depth first search algorithm and Breadth First Search algorithm and develop a recursive algorithm.

Theory

Depth First Search (DFS)

The DFS algorithm is a recursive algorithm that uses the idea of backtracking. It involves exhaustive searches of all the nodes by going ahead, if possible, else by backtracking.

Here, the word backtrack means that when you are moving forward and there are no more nodes along the current path, you move backwards on the same path to find nodes to traverse. All the nodes will be visited on the current path till all the unvisited nodes have been traversed after which the next path will be selected.

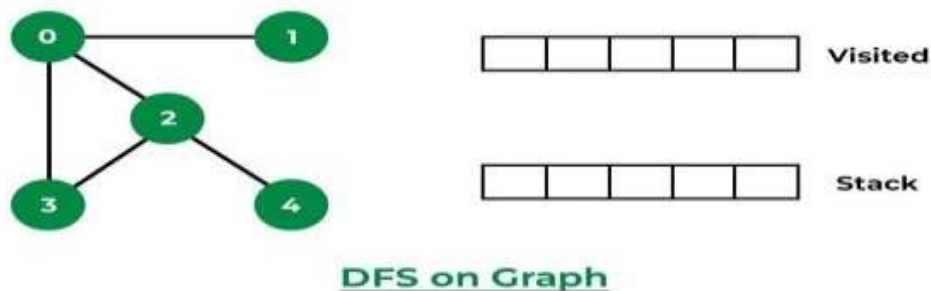
This recursive nature of DFS can be implemented using stacks. The basic idea is as follows: Pick a starting node and push all its adjacent nodes into a stack. Pop a node from stack to select the next node to visit and push all its adjacent nodes into a stack. Repeat this process until the stack is empty. However, ensure that the nodes that are visited are marked. This will prevent you from visiting the same node more than once. If you do not mark the nodes that are visited and you visit the same node more than once, you may end up in an infinite loop.

How does DFS work?

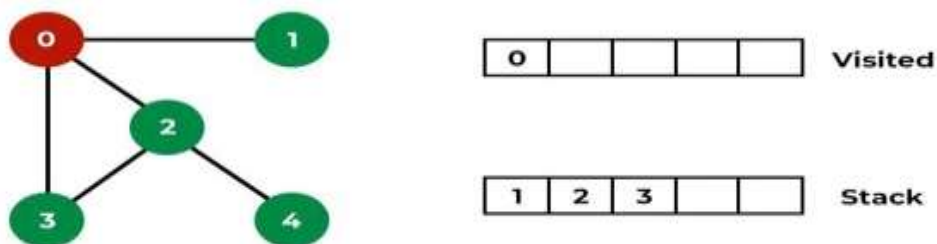
Depth-first search is an algorithm for traversing or searching tree or graph data structures. The algorithm starts at the root node (selecting some arbitrary node as the root node in the case of a graph) and explores as far as possible along each branch before backtracking.

Let us understand the working of Depth First Search with the help of the following illustration:

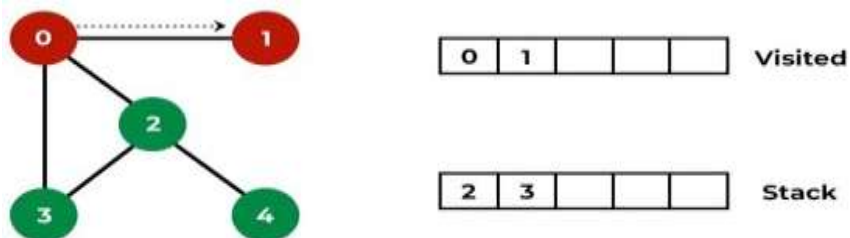
Step1: Initially stack and visited arrays are empty.



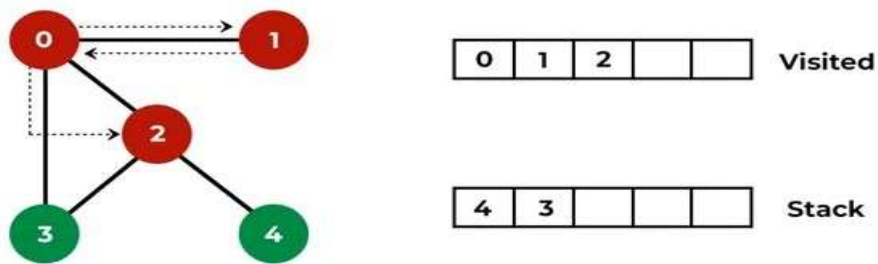
Step 2: Visit 0 and put its adjacent nodes which are not visited yet into the stack.



Step 3: Now, Node 1 at the top of the stack, so visit node 1 and pop it from the stack and put all of its adjacent nodes which are not visited in the stack.

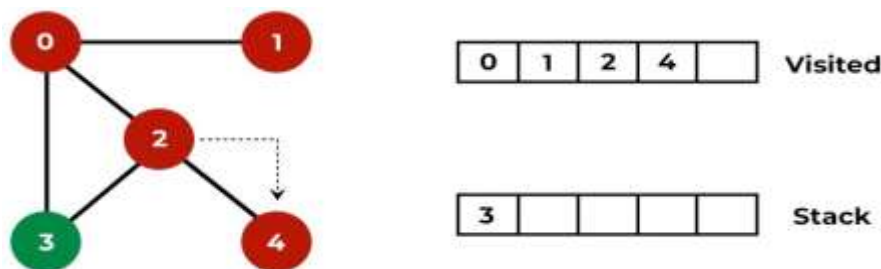


Step 4: Now, Node 2 at the top of the stack, so visit node 2 and pop it from the stack and put all of its adjacent nodes which are not visited (i.e, 3, 4) in the stack.

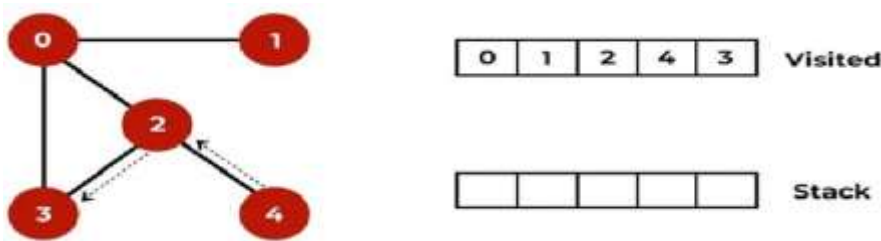


Step 5: Now, Node 4 at the top of the stack, so visit node 4 and pop it from the stack and put all of its

adjacent nodes which are not visited in the stack.



Step 6: Now, Node 3 at the top of the stack, so visit node 3 and pop it from the stack and put all of its adjacent nodes which are not visited in the stack.



Depth first search (DFS) algorithm starts with the initial node of the graph G, and then goes to deeper and deeper until we find the goal node or the node which has no children. The algorithm, then backtracks from the dead end towards the most recent node that is yet to be completely unexplored.

The data structure which is being used in DFS is stack. The process is similar to BFS algorithm. In DFS, the edges that lead to an unvisited node are called discovery edges while the edge that leads to an already visited node are called block edges.

DFS Algorithm

The recursive method of the Depth-First Search algorithm is implemented using stack. A standard Depth-First Search implementation puts every vertex of the graph into one in all 2 categories:

- 1) Visited
- 2) Not Visited.

The only purpose of this algorithm is to visit all the vertex of the graph avoiding cycles.

The DSF algorithm follows as:

1. We will start by putting any one of the graph's vertex on top of the stack.
 2. After that take the top item of the stack and add it to the visited list of the vertex.
 3. Next, create a list of that adjacent node of the vertex. Add the ones which aren't in the visited list of vertexes to the top of the stack.
 4. Lastly, keep repeating steps 2 and 3 until the stack is empty
- **Step 1:** SET STATUS = 1 (ready state) for each node in G
 - **Step 2:** Push the starting node A on the stack and set its STATUS = 2 (waiting state)

- **Step 3:** Repeat Steps 4 and 5 until STACK is empty
- **Step 4:** Pop the top node N. Process it and set its STATUS = 3 (processed state)
- **Step 5:** Push on the stack all the neighbors of N that are in the ready state (whose STATUS = 1) and set their STATUS = 2 (waiting state)
[END OF LOOP]
- **Step 6:** EXIT

Time Complexity

The time complexity of the Depth-First Search algorithm is represented within the sort of $O(V + E)$, where V is that the number of nodes and E is that the number of edges.

Space Complexity

The space complexity of the algorithm is $O(V)$.

Applications

Depth-First Search Algorithm has a wide range of applications for practical purposes. Some of them are as discussed below:

1. For finding the strongly connected components of the graph
2. For finding the path
3. To test if the graph is bipartite
4. For detecting cycles in a graph
5. Topological Sorting
6. Solving the puzzle with only one solution.
7. Network Analysis
8. Mapping Routes
9. Scheduling a problem

Breadth First Search (BFS)

Breadth-First Search (BFS) is an algorithm used for traversing graphs or trees. Traversing means visiting each node of the graph, Breadth-First Search is a recursive algorithm to search all the vertices of a graph or a tree. **BFS in python can be implemented by using data structures like a dictionary and lists.** Breadth-First Search in tree and graph is almost the same. The only difference is that the graph may contain cycles, so we may traverse to the same node again

There are many ways to traverse graphs. BFS is the most commonly used approach.

BFS is a traversing algorithm where you should start traversing from a selected node (source or starting node) and traverse the graph layer wise thus exploring the neighbor nodes (nodes which are directly connected to source node). You must then move towards the next-level neighbor nodes.

How does BFS work?

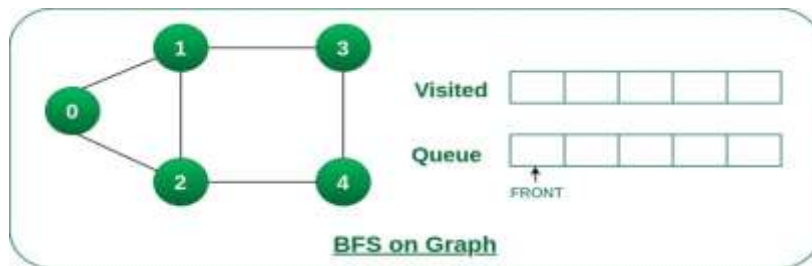
Starting from the root, all the nodes at a particular level are visited first and then the nodes of the next level are traversed till all the nodes are visited.

To do this a queue is used. All the adjacent unvisited nodes of the current level are pushed into the queue

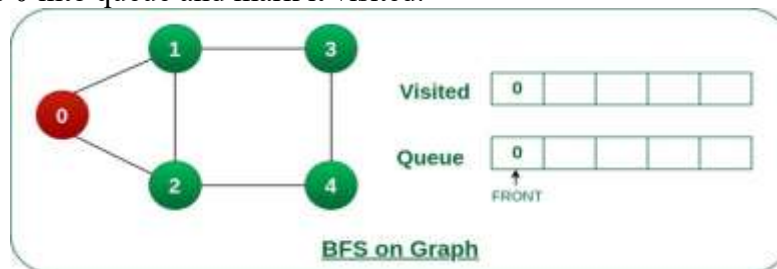
and the nodes of the current level are marked visited and popped from the queue.

Let us understand the working of the algorithm with the help of the following example:

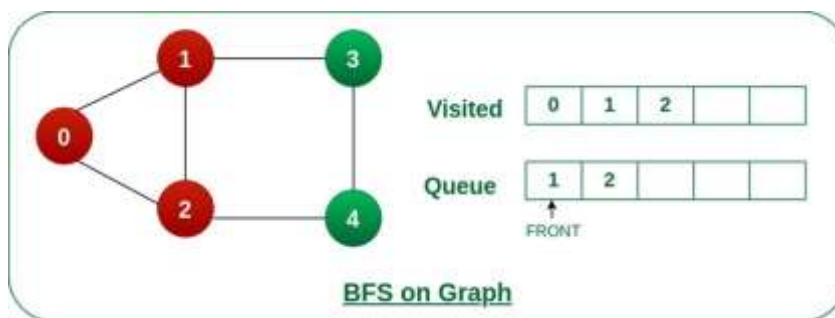
Step1: Initially queue and visited arrays are empty.



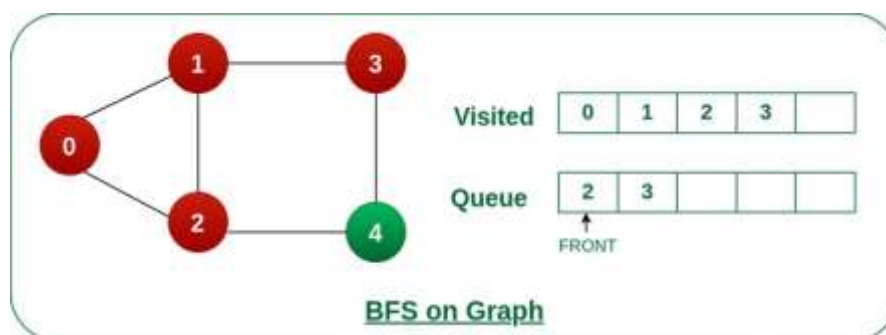
Step2: Push node 0 into queue and mark it visited.



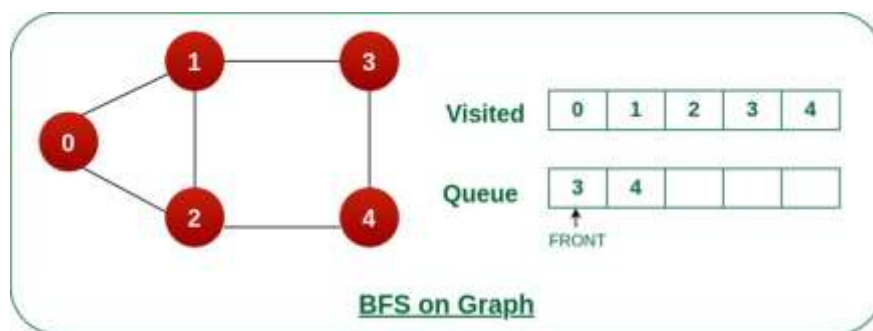
Step 3: Remove node 0 from the front of queue and visit the unvisited neighbors and push them into queue



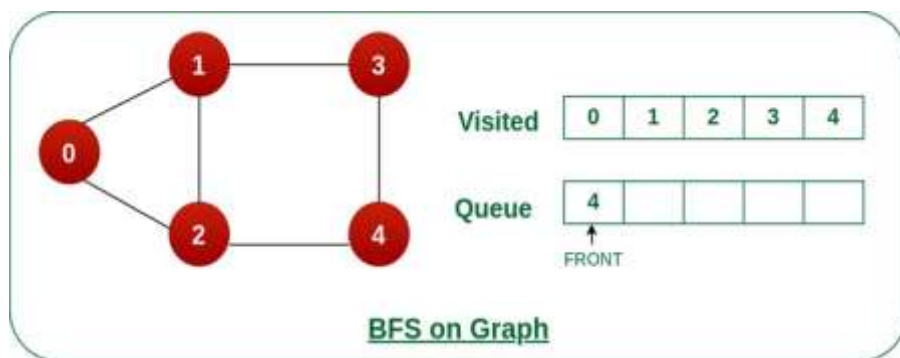
Step 4: Remove node 1 from the front of queue and visit the unvisited neighbors and push them into queue.



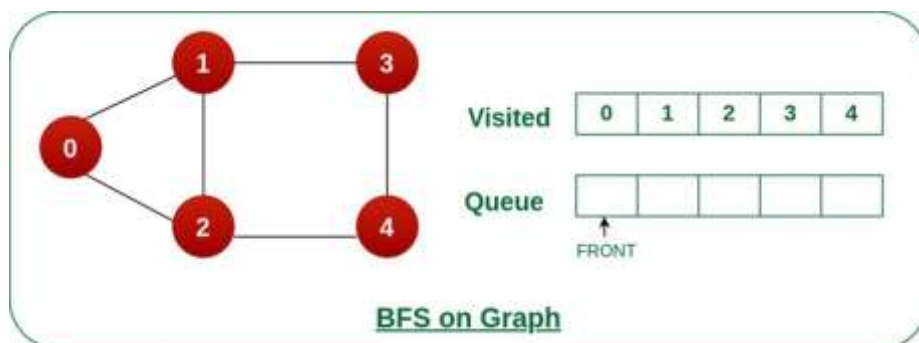
Step 5: Remove node 2 from the front of queue and visit the unvisited neighbors and push them into queue.



Step 6: Remove node 3 from the front of queue and visit the unvisited neighbors and push them into queue. As we can see that every neighbors of node 3 is visited, so move to the next node that are in the front of the queue.



Steps 7: Remove node 4 from the front of queue and visit the unvisited neighbors and push them into queue. As we can see that every neighbors of node 4 are visited, so move to the next node that is in the front of the queue.



BFS Algorithm

The steps involved in the BFS algorithm to explore a graph are given as follows –

Step 1: SET STATUS = 1 (ready state) for each node in G

Step 2: Enqueue the starting node A and set its STATUS = 2 (waiting state)

Step 3: Repeat Steps 4 and 5 until QUEUE is empty

Step 4: Dequeue a node N. Process it and set its STATUS = 3 (processed state).

Step 5: Enqueue all the neighbours of N that are in the ready state (whose STATUS = 1) and set their STATUS = 2

Step 6: EXIT

Time Complexity

The time complexity of the Breadth first Search algorithm is in the form of $O(V+E)$, where V is the representation of the number of nodes and E is the number of edges.

Also, the space complexity of the BFS algorithm is $O(V)$.

Applications

Breadth-first Search Algorithm has a wide range of applications in the real-world. Some of them are as discussed below:

1. In GPS navigation, it helps in finding the shortest path available from one point to another.
2. In path finding algorithms
3. Cycle detection in an undirected graph
4. In minimum spanning tree
5. To build index by search index
6. In Ford-Fulkerson algorithm to find maximum flow in a network

Conclusion:

Hence, we have Implemented depth first search algorithm and Breadth First Search algorithm, using an undirected graph and develop a recursive algorithm for searching all the vertices of a graph or tree data structure

Questions:

- 1) What is Depth First Search? Explain it with example?
- 2) What is Breadth First Search? Explain it with example?
- 3) Describe concept of backtracking in detail?